Release Notes

Version 5.0.0 Pre-Release.2 - October 13, 2026

General Information

In the last pre-release we said we would be moving towards a more version-control friendly GUI, and this is it. The App Designer GUI and the initializer have been **retired** and replaced by the **SPOT Console**: a browser-based interface started from MATLAB by typing SPOT, with everything technical moved into plain-text MATLAB packages (+spot) that can be read, diffed and audited like any other code. The PhaseSpace server now lives on the lab's Jetson TX2, which also hosts a second console for the cameras, the platforms' hardware, experiments, and a new Interactive Mode for flying the platforms by hand. Under the hood, the template diagram has been cleaned up (simulations now run with zero warnings), all platforms now share a single experiment clock, and the robotic arm has been redesigned around torque control. The repository has also slimmed down considerably, from 760 MB to 13 MB: new projects are created with the MATLAB function New_SPOT_Project, and the CAD, old code and old data are now kept outside of it. The consoles were put through their paces in the lab on October 9th; the arm changes that followed have not been tried on the hardware yet (see Ongoing Issues). **This version requires MATLAB R2026a or newer and Python 3.6 or newer (SPOT will offer to install Python if it can't find it). Projects made with earlier versions will not open in the new console, so create a new project and copy your custom code across. Old GUI settings files can still be imported using Load.**

Initializer Changes

- **Change 1:** Retired Run_Initializer.m. The workspace variables are now computed from the project's settings file (spot_settings.json, saved automatically on every change), and any variables of your own go in SPOT_User_Parameters.m, which runs after the settings are applied.
- **Change 2:** Replaced Setup_Experiment.m with the MATLAB function New_SPOT_Project, which copies the template into projects/ and renames the diagram after the project. It checks the name first, so it can always be used as the diagram's name.
- **Change 3:** Simulations can now be run without the GUI at all, straight from the Command Window or a script (e.g. s = spot.params.load('spot_settings.json'); spot.sim.run(s, pwd)). This makes parameter sweeps and optimization loops much simpler than manipulating the GUI handle.
- **Change 4:** The PhaseSpace data rate, the thruster PWM frequency and the lab console's address are now settings in the console rather than variables in the initializer. The arm's old gain and friction variables (Kp_sharm, Gamma, Tz_lim and friends) have been removed, as the arm model and its controllers now set these themselves.

GUI Changes

- **Change 1:** Replaced the App Designer GUI with the SPOT Console, which opens in your browser when you type SPOT in MATLAB (SPOT open reopens it, SPOT stop closes it). The page talks to a small MATLAB worker in the background, so the Command Window stays usable.
- **Change 2:** Added the Simulation tab, with the scenario, initial conditions, mass properties, LQR weights (with the resulting gains), solver and workspace in one sidebar. Simulations no longer modify or save the diagram, show their progress, and can be stopped part-way.
- **Change 3:** Redesigned the timeline so that individual phases and Phase 3 sub-phases can be switched on and off, in simulation and in experiments. Phases 0, 1 and 5 always run.
- **Change 4:** Added a shared 3D view to every tab, showing live PhaseSpace data, simulation replays and saved results side by side. Initial and home conditions are set by dragging the platforms on the table, and saved runs can be laid over a simulation (with a time offset) using Compare.
- **Change 5:** Added a Use CAD switch which draws the platforms from the lab's SolidWorks models, complete with silver and gold MLI, a plexiglass Obstacle, the arm, and the table. A ZED2 Viewpoint window shows what the Chaser's camera would see, using the camera's real intrinsics.
- **Change 6:** Renamed the platforms across the interface and the figures: RED is now the **Chaser** (silver), BLACK is the **Target** (gold) and BLUE is the **Obstacle** (blue). The code, the diagram and the saved data still use RED, BLACK and BLUE.
- **Change 7:** Added the Spacecraft Designer tab, which places docking cones (male or female), solar panels and the arm's mount on each platform. The design is shared with the lab console and drawn in every 3D view.
- **Change 8:** Added the Experiment tab. Experiments are built from your own MATLAB but compiled on the TX2, which then deploys only the program to the selected platforms (no more master/slave scheme). Start, stop and data collection are all done from the page, and the data is converted to .mat files in Saved Data automatically. Everything locks when you are off the lab network.
- **Change 9:** Added the Results tab, for loading and plotting any saved experiment or simulation (against time, or against another signal), with zoom, smoothing and linked time axes. Figures are exported by MATLAB in the lab's style from the GetStarted repository, as PDF, EPS, SVG or PNG.
- **Change 10:** Brought back the platforms' debug messages as a Messages panel, which now also works in simulation and is saved with each experiment.
- **Change 11:** Added a Live Plot tab, which plots any signal streamed from the diagram's new Live Data Streams subsystem, during simulations and experiments. A Real Time switch paces simulations to the wall clock.
- **Change 12:** When a simulation fails, the console now shows the root cause rather than Simulink's top-level message, with a Details view and a Show Block button that highlights the offending block in Simulink.

- **Change 13:** Added Load, Save As and Reset to the top bar. Load also imports old GUI state .mat files, and Reset keeps a backup of your settings before restoring the defaults.
- **Change 14:** Added the arm controllers' gains to the Control tab (when the arm is selected), and locked the data rate and thruster PWM fields behind a warning, as changing them affects the hardware.

Lab Console Changes

- **Change 1:** Moved the PhaseSpace server to the Jetson TX2 (192.168.1.116), where the SPOT Hardware Interfaces console runs at all times and starts at boot. There is no more Windows executable or password: streaming is started and stopped remotely, and each packet now carries every LED's position and the experiment clock as well as the platforms' states.
- **Change 2:** Added a Hardware tab, replacing the separate hardware web app. It turns each platform's pucks on and off, fires individual thrusters at a chosen duty cycle, and runs a thruster check. It refuses to drive a platform whose micro-controller is in use by another program, and switches everything off if the link drops.
- **Change 3:** Platforms now start experiments together. The lab console waits until every selected platform's program is running, then starts a shared countdown through the PhaseSpace stream, so all platforms agree on the experiment time to within a few milliseconds.
- **Change 4:** Added Interactive Mode to the PhaseSpace tab, which flies one or more platforms to goals you drag around the table, using the diagram's own LQR controller and thruster mixer (extracted verbatim, and checked against the diagram by spot.control.verifyConsoleController). It stops a platform if PhaseSpace loses it, it leaves the table, or the page goes quiet, and keeps goals at least 0.45 m apart.

Simulink Changes

- **Change 1:** The template diagram is now Template_v5_0_0_2026a_Jetson_PSClock.slx (saved in R2026a), and is the only diagram in the template.
- **Change 2:** Removed Custom_Library. The diagram's hardware blocks are now its own, and call System objects in the spot.hardware package, with their C/C++ code alongside them.
- **Change 3:** Replaced the Time Sync. and Check for Synchronization subsystems with an Experiment Clock, which takes the experiment time from the PhaseSpace stream. Every platform now runs on the same clock.
- **Change 4:** Replaced every NVIDIA UDP block with one UDP library of our own (spot.hardware.udpSend and udpReceive), which always hands out the newest packet rather than the oldest one in the queue.
- **Change 5:** Made the top-level subsystems atomic, with priorities, so that each step runs in a fixed order (experiment logic, then guidance, then control). This removed all 50 warnings the simulation used to print. Section 6 of the tutorial explains how this works, with a demonstration.
- **Change 6:** Rebuilt the Debugger so that each platform's messages go to the lab console on their own port, up to 120 characters long, and are also recorded in simulation.

- **Change 7:** Added a top-level Live Data Streams subsystem. Any signal connected to one of its Live Stream blocks shows up in the console's Live Plot tab.
- **Change 8:** Removed the PD controller option for the platforms (LQR only) and the drop states (initial and home conditions only).
- **Change 9:** The thruster code now runs every baseRate, and the lab console's address is now the parameter labConsoleIP rather than being hard-coded in each block.

Robotic Arm Changes

- **Change 1:** The arm is now torque-controlled throughout. The servos stay in current control, ARM_Controls are joint torques [N m], and the servos' own position and velocity laws now run as 20 Hz controllers in the diagram, with gains retuned for that rate. ARM_Control_Law_Enabler selects 1 for Default Position Control, 2 for Custom (an example which passes torques straight through) and 3 for Default Velocity Control. **Note that 3 used to mean off, so set 0 if that is what your diagram intends.**
- **Change 2:** Replaced the old Dynamixel blocks with a single Dynamixel Arm (Torque) block (spot.hardware.ArmTorque), which no longer needs the Dynamixel SDK. It reads the encoders every step and also reports the measured torques and the bus status.
- **Change 3:** Replaced the arm's dynamics blocks with a RED + ARM Plant block, which simulates the base and arm at 1 ms inside each step, with models of the servos and stick-slip friction. A full 275 s run with the arm now takes about 10 s, instead of almost two minutes.
- **Change 4:** Added a lab calibration script (resources/arm_calibration/calibrate_arm.py) which measures each joint's friction. The simulation picks up its results automatically.
- **Change 5:** Goals beyond the servos' $\pm 90^\circ$ limits (which the real servos ignore) now produce a warning in simulation, and the arm can be mounted on any face using the Spacecraft Designer.

Documentation Changes

- **Change 1:** Added a user guide, SPOT_Tutorial.pdf, which comes with every new project. It covers setup on Windows and Linux, both consoles, running without the GUI, the diagram's execution order and the arm, plus a reference of every function.
- **Change 2:** Added a GitHub wiki with a page for every function, console module, hardware program, diagram subsystem and console tab, including the maths behind the controllers, the mixer and the arm.

Bug Fixes

- **Bug 1:** Fixed a bug in the PhaseSpace server which silently dropped up to 99.7% of packets when sending to several platforms at high rates, as soon as one destination was offline.
- **Bug 2:** Fixed velocity spikes in the PhaseSpace data, which were computed from the packets' arrival times rather than PhaseSpace's own timestamps.

- **Bug 3:** Fixed the PhaseSpace receivers lagging by up to 260 ms, as they read the oldest queued packet rather than the newest one.
- **Bug 4:** Fixed the Chaser's and Target's fourth LEDs, which report as IDs 0 and 13 rather than the IDs the server expected.
- **Bug 5:** Fixed a bug in the mass properties, which divided the Target's center of gravity by the Chaser's mass.
- **Bug 6:** Fixed the Target's and Obstacle's thruster mixers, which were using the Chaser's thruster offsets and thrust.
- **Bug 7:** Fixed a bug in the synchronization check, which let any combination of platforms that included the Obstacle or the arm start without waiting for the others.
- **Bug 8:** Fixed the guidance and control running before the experiment logic within each step, which delayed every phase change by one step.
- **Bug 9:** Fixed the debug messages being cut off at 31 characters, and Phase 3 Sub-Phases 2 to 4 all reporting themselves as Sub-Phase 1.
- **Bug 10:** Fixed the arm's encoders being read at only a few Hz, which was caused by the USB adapter's 16 ms latency timer together with the Dynamixel SDK's timeout.
- **Bug 11:** Fixed the length of the arm's last link when the GUI computed the arm's position, which used b2+b3 rather than a3+b3.

Ongoing Issues

- **Issue 1:** The arm's torque control and its new hardware block have not been tested on the real arm yet. The friction values are placeholders until the calibration script is run in the lab, and the controller gains may need retuning afterwards.
- **Issue 2:** Nothing has been tested on Windows yet, including starting the console and installing Python from SPOT.
- **Issue 3:** The Spacecraft Designer is a first version: the attachments don't affect the mass properties or the 2D figures yet, and the Experiment tab draws the project's design rather than the lab console's.
- **Issue 4:** Interactive Mode flies each platform straight at its goal, without any knowledge of the others, so their paths can still cross. Keep a hand near Esc.
- **Issue 5:** Wi-Fi power saving is still enabled on the platforms, which can delay their messages to the lab console.
- **Issue 6:** A few features of the old GUI have not been brought across yet: emptying the platforms' MATLAB_ws and ExperimentSoftware folders, the vision computer, deploying to one platform at a time, video export, and custom drawings.

Acknowledgments

Special thanks to the following people who assisted with the above software changes: